# Java Task

# Agenda

01

Which is thread safety from collections?

02

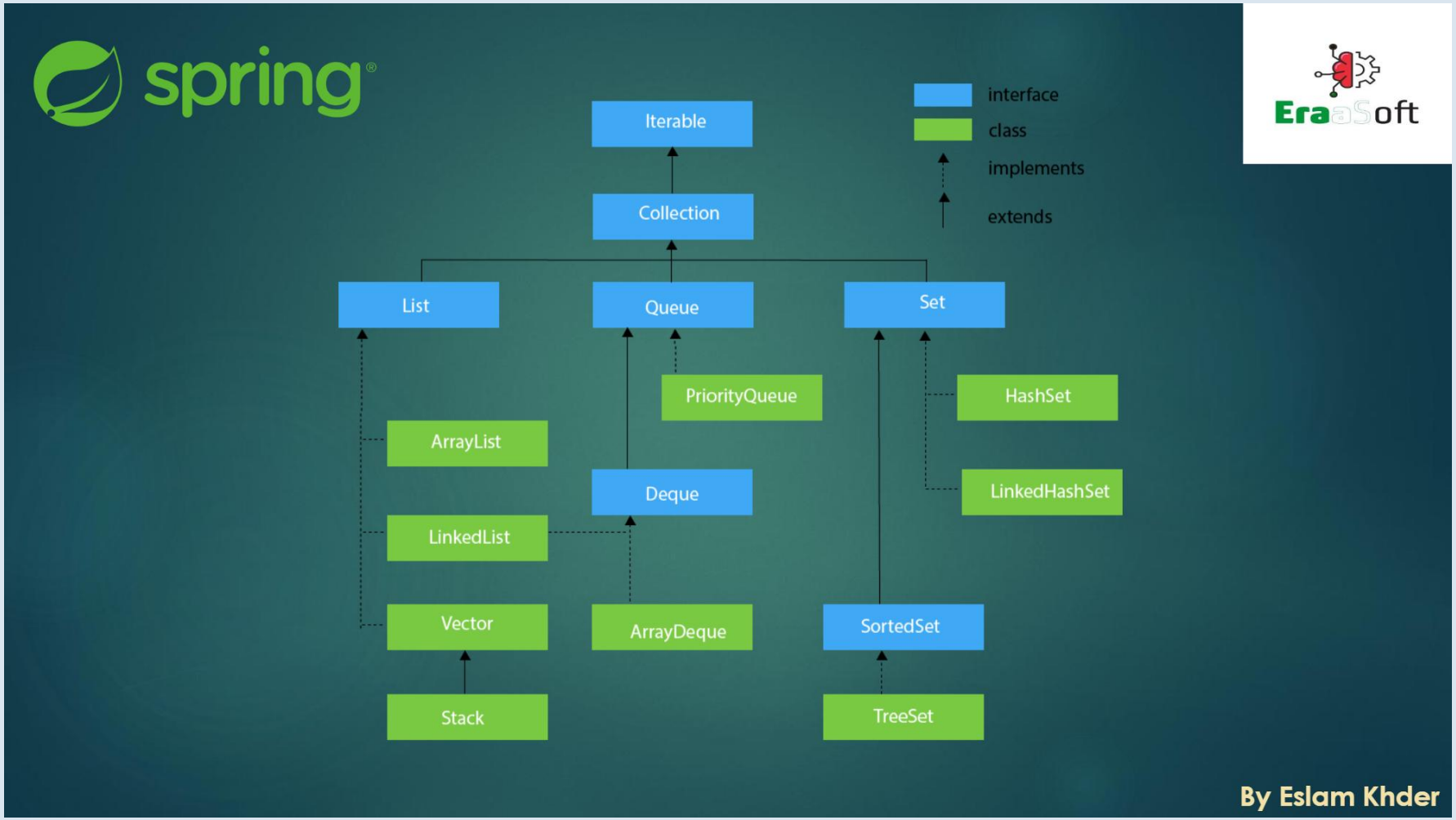
Compare between Vectors and ArrayList.

03

Compare between ArrayDequeue and Stack.

# Java collections

# Java collections



# Thread safety collections

# What is meant by thread safety in Java?

It means that a piece of code (a class, method, or data structure) works correctly when **multiple threads execute it at the same time**, without causing data corruption, unexpected behavior, or crashes.

# Thead safe collections

Vectors ➔ it’s synchronized

Stack ➔ it extends vectors

# Vector Example

```
import java.util.Vector;

public class VectorExample {

    public static void main(String[] args) {

Vector<Integer> threadSafeNumbers = new
Vector<>();

Runnable task = () -> {

            for (int i = 0; i < 5; i++) {

                threadSafeNumbers.add(i);

            }

        };

        Thread t1 = new Thread(task);

        Thread t2 = new Thread(task);

t1.start();

t2.start();

try {

        t1.join();

        t2.join();

} catch (InterruptedException e) {
e.printStackTrace();

}

System.out.println("Vector size: " +
threadSafeNumbers.size());

 }

 }
```

# Not thread safe collections

| 01 | 02 | 03 | 04 | 05 | 06 | 07 |
|---|---|---|---|---|---|---|
| ArrayList | LinkedList | HashSet | LinkedHashSet | TreeSet | PriorityQueue | ArrayDeque |

**You can make them thread-safe by using a synchronized list from collections**
**But with conditions**

# Vectors
# VS
# ArrayList

# Compare between vectors and ArrayList

| Feature | ArrayList | Vector |
|---|---|---|
| Synchronization | Not synchronized (Not thread-safe) | Synchronized (Thread-safe) |
| Performance | Faster (No locking overhead) | Slower (Overhead of synchronization) |
| Legacy Status | Modern | Legacy |
| Preferred Use | Default choice for non-threaded tasks | Rarely used in modern Java |

# Vectors Example

```
import java.util.Vector;

public class VectorExample {
    public static void main(String[] args) {
        Vector<Integer> threadSafeNumbers = new Vector<>();
         }
}
```

# ArrayList Example

```
import java.util.ArrayList;

public class ArrayListExample {
    public static void main(String[] args){
        ArrayList<String> fruits = new ArrayList<>();
    }
}
```

# Stack
# VS
# ArrayDequeue

# Compare between Stack and ArrayDequeue

| Feature | Stack | ArrayDeque |
|---|---|---|
| Inheritance | Extends Vector (Legacy) | Implements Deque (Modern) |
| Synchronization | Synchronized (Thread-safe) | Not Synchronized (Not thread-safe) |
| Performance | Slower (Overhead of locking) | Faster (No locking overhead) |
| Flexibility | Limited to Stack operations | Highly flexible (Stack + Queue + Deque) |

# Using ArrayDequeue as a Stack

```
import java.util.ArrayDeque;
import java.util.Deque;
public class StackUsageExample {
    public static void main(String[] args) {
        Deque<String> stack = new ArrayDeque<>();
}
}
```

# Thank you

Ahmed hegazy mohamed